



Wholly owned by UTAR Education Foundation
(Co. No. 578227-M)
DU012(A)

UNIVERSITI TUNKU ABDUL RAHMAN

FACULTY OF INFORMATION AND COMMUNICATION TECHNOLOGY

YEAR AND SEMESTER: 2024/06

UCCN3004 SERVER MANAGEMENT AND ADMINISTRATION

Lecturer: Dr. Fityanul Akhyar

Individual Assignment

Student Name	Student ID	No. Item	Mark (%)
Wong Pei Kei	2207466	Introduction, objective	
		Detailed 6 steps procedures and explanation	
		Conclusion	
		Report Organization & Clearness	

Assignment Task

A company has a plan to setup the Data Warehouse on Ubuntu server to store, process, and analyze large volumes of structured and unstructured data. Due to this the IT staff need detailed guide.

Table Content	Page
Introduction	3
Objective	3
Detailed 6 steps proceduces and explanation	4 - 19
Conclusion	20

Introduction

In today's data-driven environment, the ability to efficiently store, process, and analyze large volumes of data is crucial for organizations aiming to gain insights and maintain a competitive edge. Traditional data processing methods often fall short in handling the scale and complexity of modern data, which can include both structured and unstructured formats. This challenge has led to the widespread adoption of more advanced data management platforms, such as Hadoop.

Hadoop is an open-source framework that provides a reliable and scalable way to manage large datasets. It enables distributed storage and processing of data across clusters of computers, making it possible to handle big data applications with ease. The Hadoop ecosystem includes various components, such as the Hadoop Distributed File System (HDFS) for storage, and MapReduce and YARN for processing.

This guide aims to provide IT staff with a comprehensive, step-by-step approach to set up Hadoop on an Ubuntu server. The process includes installing the necessary prerequisites, downloading and configuring Hadoop, formatting the Hadoop filesystem, starting Hadoop services, and verifying the installation. By following this guide, the IT team will be equipped to deploy a powerful data warehouse solution that can meet the organization's data processing needs, ensuring scalability, reliability, and efficiency.

Objective

The objective of this guide is to assist IT staff in setting up Hadoop on an Ubuntu server, including:

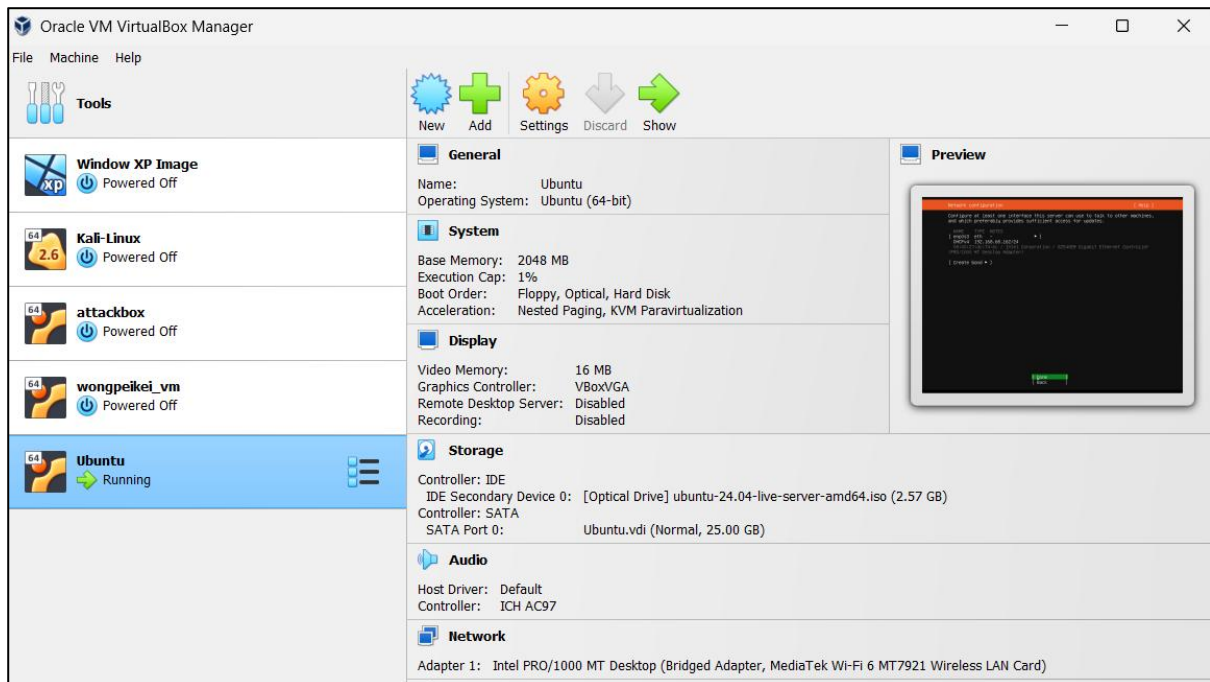
- Installing prerequisites for Hadoop
- Downloading and installing Hadoop
- Configuring Hadoop
- Formatting the Hadoop filesystem
- Starting Hadoop services
- Verifying the Hadoop installation

Detailed Steps and Explanation

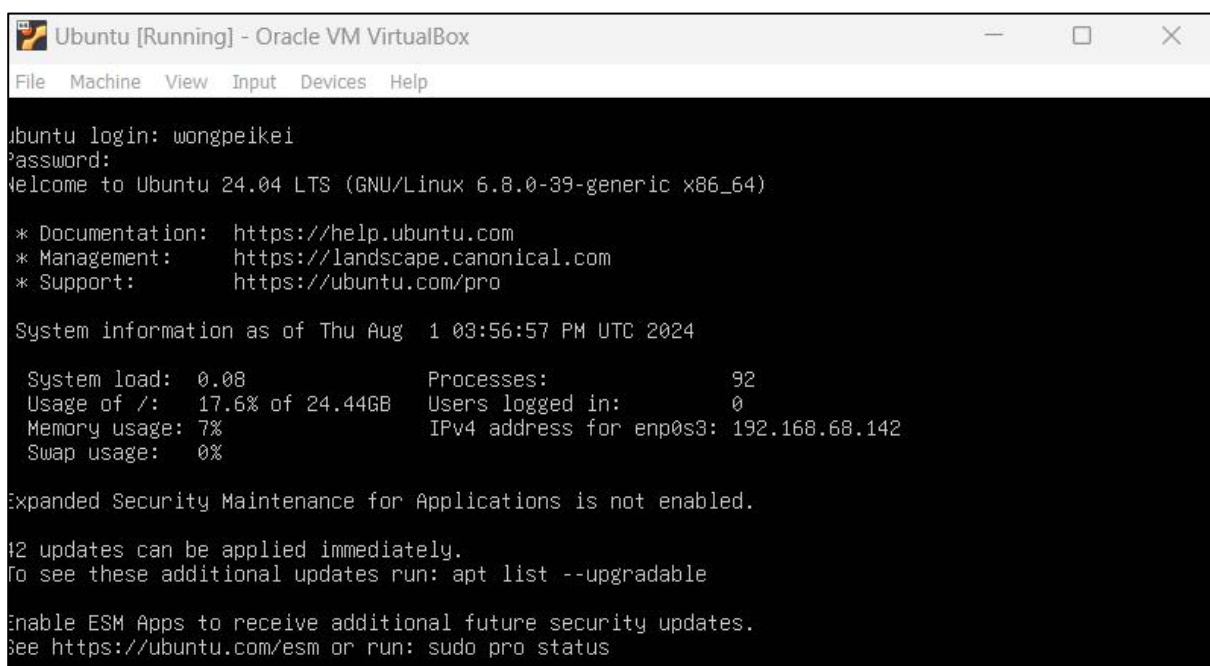
Step 1 – Hodoop Prerequisites Installation

1. Open Ubuntu in VM

Ensure you have a running Ubuntu virtual machine or physical server. This guide assumes the use of a virtual machine for ease of management and testing, but the steps are applicable to physical servers as well.



As soon as the virtual machine boots up, The VM should open directly to a terminal interface.



2. Update Ubuntu System

Ensure that your system has the latest security patches and software updates. First update the package lists and upgrade all installed packages to the latest versions by running this command:

```
sudo apt update && sudo apt upgrade -y
```

```
wongpeikei@ubuntu:~$ sudo apt update && sudo apt upgrade -y
Hit:1 http://my.archive.ubuntu.com/ubuntu noble InRelease
Hit:2 http://my.archive.ubuntu.com/ubuntu noble-updates InRelease
Hit:3 http://my.archive.ubuntu.com/ubuntu noble-backports InRelease
Hit:4 http://security.ubuntu.com/ubuntu noble-security InRelease
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
3 packages can be upgraded. Run 'apt list --upgradable' to see them.
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
Calculating upgrade... Done
The following upgrades have been deferred due to phasing:
  dracut-install libopeniscsiusr open-iscsi
0 upgraded, 0 newly installed, 0 to remove and 3 not upgraded.
wongpeikei@ubuntu:~$
```

3. Install Java

As Hadoop requires Java to run, installing Java is a critical step in preparing your environment for Hadoop. You need to install OpenJDK 11 by this command:

```
sudo apt install openjdk-11-jdk -y
```

```
wongpeikei@ubuntu:~$ sudo apt install openjdk-11-jdk -y
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
openjdk-11-jdk is already the newest version (11.0.24+8-1ubuntu3~24.04.1).
0 upgraded, 0 newly installed, 0 to remove and 2 not upgraded.
wongpeikei@ubuntu:~$ _
```

To verify Java Installation, look into the Java version to confirm the installation.

```
java -version
```

```
wongpeikei@ubuntu:~$ java -version
openjdk version "11.0.24" 2024-07-16
OpenJDK Runtime Environment (build 11.0.24+8-post-Ubuntu-1ubuntu324.04.1)
OpenJDK 64-Bit Server VM (build 11.0.24+8-post-Ubuntu-1ubuntu324.04.1, mixed mode, sharing)
wongpeikei@ubuntu:~$
```

4. Install SSH

Hadoop requires SSH for its daemons to communicate. Install the SSH server and client:

```
sudo apt install openssh-server openssh-client -y
```

```
wongpeikei@ubuntu:~$ sudo apt install openssh-server openssh-client -y
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
openssh-client is already the newest version (1:9.6p1-3ubuntu13.4).
openssh-client set to manually installed.
The following additional packages will be installed:
  libwrap0 ncurses-term openssh-sftp-server ssh-import-id
Suggested packages:
  molly-guard monkeysphere ssh_askpass
The following NEW packages will be installed:
  libwrap0 ncurses-term openssh-server openssh-sftp-server ssh-import-id
0 upgraded, 5 newly installed, 0 to remove and 2 not upgraded.
Need to get 880 kB of archives.
After this operation, 6,857 kB of additional disk space will be used.
Get:1 http://my.archive.ubuntu.com/ubuntu noble-updates/main amd64 openssh-sftp-server amd64 1:9.6p1-3ubuntu13.4 [37.4 kB]
7% [Working]_
```

5. Create a new user

Create a dedicated user for running Hadoop and grant the 'hadoop' user administrative privileges:

```
sudo adduser hadoop
```

```
sudo usermod -aG sudo hadoop
```

```
wongpeikei@ubuntu:~$ sudo adduser hadoop
info: Adding user `hadoop' ...
info: Selecting UID/GID from range 1000 to 59999 ...
info: Adding new group `hadoop' (1001) ...
info: Adding new user `hadoop' (1001) with group `hadoop (1001)' ...
info: Creating home directory `/home/hadoop' ...
info: Copying files from `/etc/skel' ...
New password:
Retype new password:
passwd: password updated successfully
Changing the user information for hadoop
Enter the new value, or press ENTER for the default
  Full Name []: WPK_DataWarehouse
  Room Number []:
  Work Phone []:
  Home Phone []:
  Other []:
Is the information correct? [Y/n] y
info: Adding new user `hadoop' to supplemental / extra groups `users' ...
info: Adding user `hadoop' to group `users' ...
wongpeikei@ubuntu:~$ sudo usermod -aG sudo hadoop
```

6. Switch to Hadoop user

```
su - hadoop
```

```
wongpeikei@ubuntu:~$ su - hadoop
Password:
To run a command as administrator (user "root"), use "sudo <command>".
See "man sudo_root" for details.

hadoop@ubuntu:~$ _
```

7. Generate SSH Key for Hadoop User

Generate an SSH key pair for the 'hadoop' user and add the public key to the authorized keys:

```
ssh-keygen -t rsa -P "" -f ~/.ssh/id_rsa

cat ~/.ssh/id_rsa.pub >> ~/.ssh/authorized_keys

chmod 600 ~/.ssh/authorized_keys
```

```
hadoop@ubuntu:~$ ssh-keygen -t rsa -P "" -f ~/.ssh/id_rsa
Generating public/private rsa key pair.
Created directory '/home/hadoop/.ssh'.
Your identification has been saved in /home/hadoop/.ssh/id_rsa
Your public key has been saved in /home/hadoop/.ssh/id_rsa.pub
The key fingerprint is:
SHA256:98DacjZz1MgkmSAWo4BMtLNiSfw6WZu+4XYkdmdZ9c8 hadoop@ubuntu
The key's randomart image is:
+----[RSA 3072]-----+
|*O =..|
|00.O 0 . 0 .|
|=. + 0 .|
|. = * . 0|
|. + 0 S * 0 0|
|O +000 + + + E|
|+.++ 0 0 * 0|
|+... + +|
|. +0|
+----[SHA256]-----+
hadoop@ubuntu:~$ sudo cat ~/.ssh/id_rsa.pub >> ~/.ssh/authorized_keys
[sudo] password for hadoop:
hadoop@ubuntu:~$ sudo chmod 600 ~/.ssh/authorized_keys
hadoop@ubuntu:~$ _
```

8. Login SSH Local User as Hadoop user without password

Test the SSH setup by logging in to localhost, you should be able to log in without a password prompt.

```
ssh localhost
```

```
hadoop@ubuntu:~$ ssh localhost
The authenticity of host 'localhost (127.0.0.1)' can't be established.
ED25519 key fingerprint is SHA256:Lj3SUpzdrsn8s7kXyQIlv72ti7b0aASESqgIDVWeMRU.
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added 'localhost' (ED25519) to the list of known hosts.
Welcome to Ubuntu 24.04 LTS (GNU/Linux 6.8.0-39-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/pro

System information as of Fri Aug  2 01:32:15 PM UTC 2024

System load:  0.02               Processes:            104
Usage of /:   19.7% of 24.44GB   Users logged in:     1
Memory usage: 12%               IPv4 address for enp0s3: 192.168.68.142
Swap usage:   0%

Expanded Security Maintenance for Applications is not enabled.

0 updates can be applied immediately.

Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status

The programs included with the Ubuntu system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Ubuntu comes with ABSOLUTELY NO WARRANTY, to the extent permitted by
applicable law.

hadoop@ubuntu:~$
```

Step 2 – Download and Install Hadoop

This phase involves obtaining the Hadoop software and setting it up on the system:

1. Download Hadoop Tarball

Use the `wget` command to download the Hadoop tarball from the Apache website. This command fetches the specified file from the internet and saves it to your current directory.

```
wget https://downloads.apache.org/hadoop/common/hadoop-3.4.0/hadoop-3.4.0.tar.gz
```

```
hadoop@ubuntu:~$ wget https://downloads.apache.org/hadoop/common/hadoop-3.4.0/hadoop-3.4.0.tar.gz
--2024-08-02 13:33:28-- https://downloads.apache.org/hadoop/common/hadoop-3.4.0/hadoop-3.4.0.tar.gz
Resolving downloads.apache.org (downloads.apache.org)... 135.181.214.104, 88.99.208.237, 2a01:4f9:3a
:2c57::2, ...
Connecting to downloads.apache.org (downloads.apache.org)|135.181.214.104|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 965537117 (921M) [application/x-gzip]
Saving to: 'hadoop-3.4.0.tar.gz'

hadoop-3.4.0.tar.gz      100%[=====>] 920.81M  3.35MB/s   in 5m 57s
2024-08-02 13:39:27 (2.58 MB/s) - 'hadoop-3.4.0.tar.gz' saved [965537117/965537117]

hadoop@ubuntu:~$
```

2. Extract Downloaded Hadoop

Use the `tar` command to extract the contents of the downloaded tarball.

```
hadoop@ubuntu:~$ tar -xvzf hadoop-3.4.0.tar.gz
```

3. Move Extracted Files

Move the extracted Hadoop files to the `/usr/local` directory for easier management and standardization. `/usr/local/hadoop` is the target directory for Hadoop installation.

```
hadoop@ubuntu:~$ sudo mv hadoop-3.4.0 /usr/local/hadoop
[sudo] password for hadoop:
hadoop@ubuntu:~$
```

4. Configure Environment Variables

Open the `.bashrc` file to add Hadoop environment variables. The `.bashrc` file is a script that runs whenever a new terminal session is started in interactive mode. Editing this file allows us to set environment variables.

```
hadoop@ubuntu:~$ nano ~/.bashrc_
```

Add the following lines to the end of the `.bashrc` file. These lines set up the necessary environment variables for Hadoop.

```

GNU nano 7.2 /home/hadoop/.bashrc
# Add an "alert" alias for long running commands.  Use like so:
#   sleep 10; alert
alias alert='notify-send --urgency=low -i "${ $? = 0 }" && echo terminal || echo error)' "${history}

# Alias definitions.
# You may want to put all your additions into a separate file like
# ~/.bash_aliases, instead of adding them here directly.
# See /usr/share/doc/bash-doc/examples in the bash-doc package.

if [ -f ~/.bash_aliases ]; then
    . ~/.bash_aliases
fi

# enable programmable completion features (you don't need to enable
# this, if it's already enabled in /etc/bash.bashrc and /etc/profile
# sources /etc/bash.bashrc).
if ! shopt -oq posix; then
    if [ -f /usr/share/bash-completion/bash_completion ]; then
        . /usr/share/bash-completion/bash_completion
    elif [ -f /etc/bash_completion ]; then
        . /etc/bash_completion
    fi
fi

export HADOOP_HOME=/usr/local/hadoop
export HADOOP_INSTALL=$HADOOP_HOME
export HADOOP_MAPRED_HOME=$HADOOP_HOME
export HADOOP_COMMON_HOME=$HADOOP_HOME
export HADOOP_HDFS_HOME=$HADOOP_HOME
export YARN_HOME=$HADOOP_HOME
export HADOOP_COMMON_LIB_NATIVE_DIR=$HADOOP_HOME/lib/native
export PATH=$PATH:$HADOOP_HOME/sbin:$HADOOP_HOME/bin

hadoop@ubuntu:~$ _

```

Reload the *.bashrc* file to apply the changes made to the environment variables.

```

hadoop@ubuntu:~$ source ~/.bashrc
hadoop@ubuntu:~$ _

```

Step 3 – Hadoop Configuration

Configuring Hadoop involves setting up environment variables, core Hadoop settings, HDFS settings, MapReduce, and YARN.

1. Configure Hadoop Environment

First, change the directory to the Hadoop configuration directory.

```
cd /usr/local/hadoop/etc/hadoop
```

```
hadoop@ubuntu:/$ cd /usr/local/hadoop/etc/hadoop
hadoop@ubuntu:/usr/local/hadoop/etc/hadoop$ ls
capacity-scheduler.xml      httpfs-env.sh              mapred-site.xml
configuration.xml           httpfs-log4j.properties   shellprofile.d
container-executor.cfg      httpfs-site.xml           ssl-client.xml.example
core-site.xml               kms-acls.xml              ssl-server.xml.example
hadoop-env.cmd              kms-env.sh                user_ec_policies.xml.template
hadoop-env.sh               kms-log4j.properties      workers
hadoop-metrics2.properties kms-site.xml              yarn-env.cmd
hadoop-policy.xml           log4j.properties          yarn-env.sh
hadoop-user-functions.sh.example mapred-env.cmd            yarnservice-log4j.properties
hdfs-rbf-site.xml           mapred-env.sh             yarn-site.xml
hdfs-site.xml               mapred-queues.xml.template
hadoop@ubuntu:/usr/local/hadoop/etc/hadoop$
```

Open the *hadoop-env.sh* file to set the 'JAVA_HOME' environment variable.

```
vi hadoop-env.sh
```

```
hadoop@ubuntu:/usr/local/hadoop/etc/hadoop$ vi hadoop-env.sh_
```

Add the following line to specify the path to the Java installation.

JAVA_HOME → environment variable specifying the location of the Java installation

```
# Unless required by applicable law or agreed to in writing, software
# distributed under the License is distributed on an "AS IS" BASIS,
# WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
# See the License for the specific language governing permissions and
# limitations under the License.

# Set Hadoop-specific environment variables here.
export JAVA_HOME=/usr/lib/jvm/java-11-openjdk-amd64

## THIS FILE ACTS AS THE MASTER FILE FOR ALL HADOOP PROJECTS.
## SETTINGS HERE WILL BE READ BY ALL HADOOP COMMANDS.  THEREFORE,
## ONE CAN USE THIS FILE TO SET YARN, HDFS, AND MAPREDUCE
## CONFIGURATION OPTIONS INSTEAD OF xxx-env.sh.
##
## Precedence rules:
## {yarn-env.sh|hdfs-env.sh} > hadoop-env.sh > hard-coded defaults
##
## {YARN_xyz|HDFS_xyz} > HADOOP_xyz > hard-coded defaults
##

# Many of the options here are built from the perspective that users
# may want to provide OVERWRITING values on the command line.
# For example:
#
# "hadoop-env.sh" 434L, 16836B written
hadoop@ubuntu:/usr/local/hadoop/etc/hadoop$
```

2. Configure Core Hadoop Settings

Open the *core-site.xml* file to configure core Hadoop settings.

```
vi core-site.xml
```

```
hadoop@ubuntu:~/hadoop/etc/hadoop$ vi core-site.xml
```

Add the following configuration to set the default file system.

<configuration> → root element for Hadoop configuration

<property> → defines a configuration property

fs.defaultFS → property name specifying the default file system URI

hdfs://localhost:9000 → URI of the Hadoop file system

```
<configuration>
  <property>
    <name>fs.defaultFS</name>
    <value>hdfs://localhost:9000</value>
  </property>
</configuration>
```

```
"core-site.xml" 24L, 866B written
hadoop@ubuntu:/usr/local/hadoop/etc/hadoop$
```

3. Configure HDFS Settings

Open the *hdfs-site.xml* file to configure HDFS settings.

```
vi hdfs-site.xml
```

```
hadoop@ubuntu:~/hadoop/etc/hadoop$ vi hdfs-site.xml
```

Add the following configuration to specify HDFS properties.

dfs.replication → property specifying the default block replication factor

dfs.namenode.name.dir → directory for storing the NameNode data

dfs.datanode.data.dir → directory for storing the DataNode data

```
<configuration>
  <property>
    <name>dfs.replication</name>
    <value>1</value>
  </property>
  <property>
    <name>dfs.namenode.name.dir</name>
    <value>file:///usr/local/hadoop/hadoop_data/hdfs/namenode</value>
  </property>
  <property>
    <name>dfs.datanode.data.dir</name>
    <value>file:///usr/local/hadoop/hadoop_data/hdfs/datanode</value>
  </property>
</configuration>
"hdfs-site.xml" 32L, 1109B written
hadoop@ubuntu:/usr/local/hadoop/etc/hadoop$ _
```

Create directories for storing NameNode and DataNode data.

```
hadoop@ubuntu:~$ mkdir -p /usr/local/hadoop/hadoop_data/hdfs/namenode
hadoop@ubuntu:~$ mkdir -p /usr/local/hadoop/hadoop_data/hdfs/datanode
hadoop@ubuntu:~$ sudo chown -R hadoop:hadoop /usr/local/hadoop
[sudo] password for hadoop:
hadoop@ubuntu:~$ _
```

4. Configure MapReduce

Open the *mapred-site.xml* file to configure MapReduce settings.

```
vi mapred-site.xml
```

```
hadoop@ubuntu:~/hadoop/etc/hadoop$ vi mapred-site.xml
```

Add the following configuration to specify the MapReduce framework.

mapreduce.framework.name → property specifying framework used for MapReduce

yarn → specifies that YARN is used as the resource management framework.

```
<configuration>
  <property>
    <name>mapreduce.framework.name</name>
    <value>yarn</value>
  </property>
</configuration>
```

```
"mapred-site.xml" 24L, 844B written
hadoop@ubuntu:~/usr/local/hadoop/etc/hadoop$
```

5. Configure YARN Settings

Open the *yarn-site.xml* file to configure YARN settings.

```
vi yarn-site.xml
```

```
hadoop@ubuntu:~/hadoop/etc/hadoop$ vi yarn-site.xml_
```

Add the following configuration to specify YARN properties.

yarn.nodemanager.aux-services → property specifying auxiliary services for the NodeManager

mapreduce_shuffle → specifies the MapReduce shuffle service

```
<configuration>
<!-- Site specific YARN configuration properties -->
  <property>
    <name>yarn.nodemanager.aux-services</name>
    <value>mapreduce_shuffle</value>
  </property>
</configuration>

```

"yarn-site.xml" 22L, 794B written
hadoop@ubuntu:/usr/local/hadoop/etc/hadoop\$ _

Step 4 – Format the Hadoop Filesystem

1. Ensure you are logged in as the Hadoop user

Confirm you're logged in as the Hadoop user with the necessary permissions.

```
whoami
```

```
hadoop@ubuntu:/usr/local/hadoop/etc/hadoop$ whoami
hadoop
hadoop@ubuntu:/usr/local/hadoop/etc/hadoop$ _
```

2. Format the Namenode to initialize the HDFS

Set up the directory structure needed for Hadoop's file system.

```
hdfs namenode -format
```

```
hadoop@ubuntu:~/hadoop/etc/hadoop$ hdfs namenode -format
_
```

Step 5 – Start Hadoop Services

1. Start HDFS services

Start the essential HDFS services like Namenode and Datanodes for distributed storage.

```
start-dfs.sh
```

```
hadoop@ubuntu:/usr/local/hadoop/hadoop_data$ start-dfs.sh
Starting namenodes on [localhost]
Starting datanodes
Starting secondary namenodes [ubuntu]
ubuntu: Warning: Permanently added 'ubuntu' (ED25519) to the list of known hosts.
2024-08-02 14:47:39,350 WARN util.NativeCodeLoader: Unable to load native-hadoop library for your pl
atform... using builtin-java classes where applicable
hadoop@ubuntu:/usr/local/hadoop/hadoop_data$
```

2. Start YARN services

Start YARN services, including ResourceManager and NodeManagers, which manage resources and schedule tasks.

```
start-yarn.sh
```

```
hadoop@ubuntu:/usr/local/hadoop/hadoop_data$ start-yarn.sh
Starting resourcemanager
Starting nodemanagers
hadoop@ubuntu:/usr/local/hadoop/hadoop_data$
```

Step 6 – Verify Hadoop Installation

1. Check Hadoop Processes

Verify that the Hadoop services are running by checking the Java processes: *jps*

```
hadoop@ubuntu:/usr/local/hadoop/hadoop_data$ jps
11712 DataNode
11864 SecondaryNameNode
12619 Jps
12267 NodeManager
12141 ResourceManager
11599 NameNode
hadoop@ubuntu:/usr/local/hadoop/hadoop_data$
```

2. Access Hadoop Web Interfaces

Verify the Hadoop services by accessing the web interfaces:

NameNode → <http://localhost:9870>

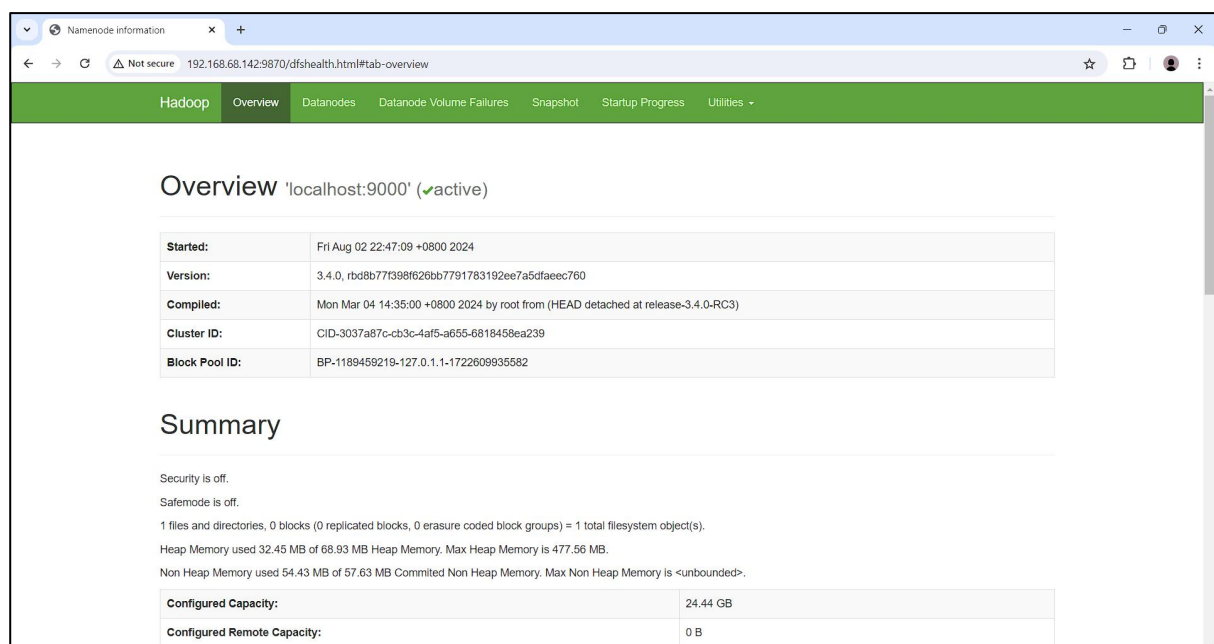
Secondary NameNode → <http://localhost:9868>

ResourceManager → <http://localhost:8088>

NodeManager → <http://localhost:8042>

Note: Replace 'localhost' with your server's actual IP address

NodeNode:



The screenshot shows the Hadoop NameNode web interface in a browser. The address bar shows the URL <http://192.168.68.142:9870/dfshealth.html#tab-overview>. The interface has a green header with tabs: Hadoop, Overview, Datanodes, Datanode Volume Failures, Snapshot, Startup Progress, and Utilities. The main content area is titled "Overview 'localhost:9000' (✓active)". Below the title is a table with the following information:

Started:	Fri Aug 02 22:47:09 +0800 2024
Version:	3.4.0, rbd8b77f398f626bb7791763192ee7a5dfaee760
Compiled:	Mon Mar 04 14:35:00 +0800 2024 by root from (HEAD detached at release-3.4.0-RC3)
Cluster ID:	CID-3037a87c-cb3c-4af5-a655-6818458ea239
Block Pool ID:	BP-1189459219-127.0.1.1-1722609935582

Below the table is a "Summary" section. It contains the following text:

Security is off.
Safemode is off.
1 files and directories, 0 blocks (0 replicated blocks, 0 erasure coded block groups) = 1 total filesystem object(s).
Heap Memory used 32.45 MB of 68.93 MB Heap Memory. Max Heap Memory is 477.56 MB.
Non Heap Memory used 54.43 MB of 57.63 MB Committed Non Heap Memory. Max Non Heap Memory is <unbounded>.

At the bottom, there is a table with the following information:

Configured Capacity:	24.44 GB
Configured Remote Capacity:	0 B

Secondary NameNode:

Namenode information

All Applications

Secondary/Namenode information

NodeManager information

192.168.68.142:9868/status.html

Hadoop

Overview

Overview

Version	3.4.0
Compiled	2024-03-04T06:35Z by root from (HEAD detached at release-3.4.0-RC3)
NameNode Address	localhost:9000
Started	Fri Aug 02 22:47:03 +0800 2024
Last Checkpoint	Fri Aug 02 22:48:40 +0800 2024 (12 mins, 22 sec ago)
Checkpoint Period	3600 seconds
Checkpoint Transactions	1000000

Checkpoint Image URI

file:///tmp/hadoop-hadoop/dfs/namesecondary

Checkpoint Editlog URI

file:///tmp/hadoop-hadoop/dfs/namesecondary

ResourceManager:

Namenode information

All Applications

192.168.68.142:8088/cluster



Cluster

About

Nodes

Node Labels

Applications

NEW

NEW SAVING

SUBMITTED

ACCEPTED

RUNNING

FINISHED

FAILED

KILLED

Scheduler

Tools

All Applications

Cluster Metrics

Apps Submitted	Apps Pending	Apps Running	Apps Completed	Containers Running	Used Resources
0	0	0	0	0	<memory:0 B, vCores:0>

Cluster Nodes Metrics

Active Nodes	Decommissioning Nodes	Decommissioned Nodes	Lost Nodes
1	0	0	0

Scheduler Metrics

Scheduler Type	Scheduling Resource Type	Minimum Allocation	Maximum Allocation	Maximum Cluster Application F
Capacity Scheduler	[memory-mb (unit=Mi), vcores]	<memory:1024, vCores:1>	<memory:8192, vCores:4>	0

Show 20 entries

ID	User	Name	Application Type	Application Tags	Queue	Application Priority	StartTime	LaunchTime	FinishTime	State	FinalStatus	Running Containers	Allocated CP	VCores
No data available in table														

Showing 0 to 0 of 0 entries

NodeManager:

▼

▼

▼

▼

▼

Namenode information

All Applications

SecondaryNamenode information

NodeManager information

+

192.168.68.142:8042/node



ResourceManager

NodeManager

Node Information

List of Applications

List of Containers

Tools

NodeManager information

Total Vmem allocated for Containers	16.80 GB
Vmem enforcement enabled	true
Total Pmem allocated for Container	8 GB
Pmem enforcement enabled	true
Total VCores allocated for Containers	8
Resource types	memory-mb (unit=Mi), vcores
NodeHealthyStatus	true
LastNodeHealthTime	Fri Aug 02 15:02:11 UTC 2024
NodeHealthReport	
NodeManager started on	Fri Aug 02 14:51:48 UTC 2024
NodeManager Version:	3.4.0 from bd8b77f398f626bb7791783192ee7a5dfaee760 by root source checksum 934da0c5743762b7851cfcff9f8ca2 on 2024-03-04T06:54Z
Hadoop Version:	3.4.0 from bd8b77f398f626bb7791783192ee7a5dfaee760 by root source checksum 17fe694a3613358b38812ae9c31114e on 2024-03-04T06:35Z

Conclusion

This detailed guide provides a thorough, step-by-step approach to installing and configuring Hadoop on an Ubuntu server, enabling IT staff to set up a robust and scalable data warehouse infrastructure. By carefully following the instructions, the organization can ensure that its environment is properly configured to support Hadoop's ecosystem, allowing for efficient processing, storage, and analysis of large data volumes.

The configuration phase, including setting up the core settings, HDFS, MapReduce, and YARN, is critical for optimizing the performance and reliability of the Hadoop system. Creating necessary directories and configuring environment variables ensures that Hadoop can leverage the server's resources effectively, facilitating high fault tolerance and efficient data management. This setup is essential for businesses that anticipate growth in their data needs, providing a scalable solution that can adapt to increasing data volumes without significant complexity or cost.

Overall, by establishing a well-configured Hadoop environment, the company can harness the full potential of distributed computing. This robust setup will enable the organization to transform raw data into actionable insights, driving informed business decisions and fostering innovation. The scalable and reliable data warehouse solution ensures that the company is well-equipped to handle current and future data challenges, enhancing its competitive edge in the data-driven marketplace.